

Trading Desk v3

Project overview · refreshed September 24, 2026

Build a small, auditable research desk that tests whether models, memory and carefully bounded AI assistance improve trading decisions after costs.

The idea

Start by collecting market data, training and comparing algorithms, and testing them honestly on later, unseen periods. Then use a small decision-support team to explain model signals, challenge proposals and recall relevant past outcomes. A deterministic risk gate—not an AI prompt—controls what may proceed.

Why build it?

- **Separate evidence from a good story.** Compare strategies with simple baselines, transaction costs and realistic timing before trusting them.
- **Keep continuity.** Connect a decision to its evidence, outcome and lesson so later decisions can use history without seeing the future.
- **Make decisions inspectable.** Preserve the reasoning, limits and approvals needed to understand why an action was allowed or rejected.

The intended loop



Reviewed outcomes feed future research and eligible memory. Improvements are evaluated against a fixed baseline; they are not automatically adopted.

What “learning” means here

Numerical learning

Retrain models on dated evidence and compare new candidates with the existing strategy.

Structured memory

Retrieve past lessons and propose policy improvements, with testing and human review before adoption.

Scope: daily, long/flat equities first; spot-crypto research stays separate. This is not an unrestricted AI trading bot, high-frequency platform or promise of returns. Real-money trading is optional and is not implemented in the starter.

The architecture, simplified

Six domains, shared controls and a small starting stack

The architecture has **six responsibility layers (domains)** and two shared operating functions. These are clear boundaries inside a modular application—not a requirement to build six separate services.

D1 Data and features

Collect, date and validate market observations; turn them into consistent numerical inputs.

D2 Research and model lifecycle

Train and compare algorithms, test costs and holdouts, and preserve evidence for candidate models.

D3 Decision support

Combine model signals and eligible lessons into explainable proposals. AI may assist with language, not execution authority.

D4 Portfolio and risk

Value the book and check projected exposure, loss limits, available cash and approval validity.

D5 Execution and accounting

Manage authorized orders, fills, fees and cash; reconcile the ledger with the broker. Broker integration is future work.

D6 Memory and evaluation

Connect decisions to matured outcomes, retrieve relevant lessons and measure whether adaptation actually helps.

Two functions span every layer

Control plane: human approvals, scheduling, environment selection, HALT, secrets, releases and recovery.

Evidence plane: decision history, data/model lineage, tests, metrics, alerts, audit export and backups.

The most important boundary

Models compute. Agents help explain. Risk enforces limits. Humans retain authority. Agents cannot bypass the risk gate, approve their own actions or receive direct broker submission privileges. Unknown state should stop new risk, not invite a guess.

Simple deployment first

One Python package and one writer on a laptop. Parquet stores research data; SQLite holds prototype state. LangGraph is optional for checkpoints and human pauses. Add databases, servers or cloud services only when measured needs justify them.

Target versus current: shared targets, frozen artifacts and marked internal-paper accounting exist. Independent parity, authenticated approval, broker execution and operational certification remain gates.

Portfolio-first foundations

Implemented interfaces, evidence and durable internal paper

Versioned evidence → scores → shared allocation → PortfolioTarget → fresh-mark plan → explicit approval and reservation → internal fills and events.

Allocation and explicit cash

Equal-weight and inverse-volatility allocation with optional reduce-only regime scaling. Immutable targets retain cash, decision/execution times and data/model/config lineage. Cash is not normalized into assets.

Rebalancing and exits

Plan over all holdings and targets using complete fresh marks. Include sells and exits before buys; do not spend proceeds from unfilled sales. Desired allocation is not execution authority.

Snapshots and frozen models

Content-addressed raw/feature snapshots, frozen estimators/transforms, digest/schema/code checks and explicit local model review. Approved inference has no silent fallback; stateless rules and replay are distinct modes.

Research evidence

Versioned runs, exposure/cash histories, benchmark costs, matched-exposure and cash baselines, fold dispersion and doubled-cost sensitivity. Supplied sessions and known-at membership do not make Yahoo a PIT feed.

Durable internal simulator

Marked equity, session/cashflow P&L, partial fills, buys/sells, fees, cash/share reservations, cancellation and UNKNOWN blocking. Duplicate/conflicting fills are checked; fill and event writes are transactional.

Read-only reporting

Escaped static HTML research reports and paper-order statistics. There are no dashboard trading controls or synthetic results represented as real observations.

What was adopted—and what was not

FinRL-X informed weight-centric interfaces only. No package dependency or execution code was imported. Fabricated-price fallback and cash renormalization were not adopted.

Boundary: this is an internal simulator, not an exchange or broker. Supplied marks/fills are not a venue feed; a local reviewer string is not authentication. Shared allocation does not prove research/execution equivalence.

Agents and bounded AI

Four roles, shared numeric decisions and disabled optional integrations

Four deterministic Python roles—not four autonomous LLM agents. Numeric rules and shared allocation remain separate from explanatory prose.

Model Signal

Formats the model card's score, price, features and provenance for the decision record. It does not train the model or compute a new score.

Bull Analyst

Builds favorable evidence from moving averages, momentum and RSI, with eligible historical lessons. It does not authorize execution.

Bear Analyst

Highlights adverse momentum, volatility, RSI, score, costs and fill risks, with eligible lessons. Its prose does not set size.

Portfolio Manager

Produces a structured proposal and evidence. Shared target allocation supplies desired deltas; risk/trust may reduce them. The standalone helper retains its compatibility sizing rule.

Optional language assistance stays bounded

PM refinement is disabled by default. When explicitly enabled, it rewrites only thesis and invalidation text—not score, direction, position size or risk limits. Provider/key binding, injection tests and measured usefulness remain admission work.

The separate sentiment boundary is disabled and shadow-only. It validates publication/ availability times, typed responses and request/cost metadata. No news provider is connected; its results never enter allocation or execution.

Orchestration is not authority

Direct cycle and LangGraph share decision, target and projected-risk helpers plus decision-time lesson filtering. Their legacy immediate-close fills remain different from the durable paper CLI. A checkpoint is not portfolio truth or a new authorization.

Bull/Bear arguments do not independently decide direction or size. Agents cannot approve themselves. Graph approval expires from the snapshot (300 seconds by default), with risk and HALT rechecked before the toy fill.

From idea to reliable operation

The phased roadmap and an honest view of readiness

Progress is controlled by **evidence gates, not calendar deadlines**. Research can produce a useful negative result; a model is not entitled to promotion because the tools work.

0	Reproducible laptop	Install the locked environment and prove that offline tests and the CLI work.
1	Credible research	Validate data and model experiments; admit a candidate only with evidence—or promote nothing.
2	Correct paper lifecycle	Build marked accounting, realistic paper execution, risk controls and replay-safe approvals.
3	Measured memory	Test retrieval, policy and bounded adaptation against a fixed numerical baseline.
4	Broker-paper certification	Prove reconciliation and failure handling; consider a supervised tiny-live trial only after separate approval.
5	Reliable operations	Monitor, back up, restore and manage releases; expand only through reviewed changes.

Observation floors: 20 internal-paper equity sessions in Phase 2; 20 broker-paper sessions in Phase 4; 30 calendar days of operational evidence in Phase 5. These are not proof of profit or completion estimates.

Available in the starter

- Frozen artifacts, current approved inference and research evidence.
- Shared targets and a durable marked internal-paper simulator.
- Optional graph approvals, filtered lessons and read-only reports.

Still required for operation

- Independent accounting parity and realistic venue fill semantics.
- Authentic PIT inputs, promotion policy and immutable outcomes.
- Broker reconciliation, authenticated approvals, remote audit and recovery.

What success looks like

Reproducible evidence, understandable decisions, enforced limits and dependable operations. If memory or AI does not improve the baseline, keep the simpler system. Staying paper-only—or retiring a weak strategy—is a valid outcome.

Current status: research and internal-paper prototype. No live broker adapter is provided. Tests and design documents do not establish profitable performance or production readiness.

Verification and release boundaries

Current automated coverage versus remaining admission obligations

Expanded offline regression coverage

- **Portfolio:** explicit cash, deterministic targets, invalid weights, reduce-only scaling, exits and missing/stale/future marks.
- **Artifacts:** immutable snapshots, model approval, digest/schema rejection, mature labels, supplied sessions, known-at membership and current inference.
- **Ledger:** partial fills, fees, session P&L, duplicate/conflicting IDs, cross-connection reservations, expiry/HALT checks, UNKNOWN blocking and audit rollback.
- **Decisions and paper CLI:** direct/graph helper parity, structured evidence, future/open-lesson exclusion, explicit review and no side effects from a plan.
- **Reports and shadow sentiment:** escaped read-only HTML, disabled integration, availability validation and bounded requests.

What the tests do not certify

Independent full-ledger/corporate-action parity, real liquidity and cancel/fill races, authentic PIT data, statistical alpha, authenticated/revocable approvals, broker reconciliation, remote audit and operational restore/observation remain gates. Test totals must come from a current run, not a historical count.

Reproducible development and operation

Use Python 3.12 and the existing lockfile with dev/graph extras. Run pytest, Ruff and CLI help offline before networked research. New interfaces added no application dependency. Coding-agent tooling is optional and separate from application authority.

Use dedicated data/config profiles. The durable simulator follows session → plan → approve → fill → book. HALT blocks new risk; it is not a broker flatten command. Do not delete a database to resolve uncertain state or treat restore as permission to resume.

Release boundary: No live broker adapter is provided. Local hashes detect corruption, not replacement by someone controlling the artifact store. Load trusted local artifacts only; keep observation, independent review and recovery evidence separate from implementation completion.